

SEMINAR SUMMARY & STUDY COMPANION

Securing MERN applications

A seminar summary and course guide for B.Tech students.

The session used Common, a MERN social app, to explain how everyday features such as profiles, comments and login can become insecure, and where the server must enforce the rules.

Verify identity, enforce permission, validate input and control the response.

Follow the request

React -> HTTP request -> Express authentication and permission checks -> validated database operation -> selected response fields -> safe React rendering. Browser input is untrusted, even when it comes from your own UI.

The key distinctions

- Authentication: who is making the request? Authorization: may this person perform this action on this resource?
- Confidentiality protects private data; integrity protects permitted changes; availability keeps the service usable.
- A cookie carries a value. A server session ID identifies server-held state. A signed JWT contains verifiable claims; its payload is not secret by default.
- HttpOnly limits JavaScript access to cookies. Secure requires HTTPS. SameSite affects when cookies are sent.
- Validation checks input shape. Safe rendering keeps content from becoming active browser code.

The Common reference app

The published app includes the demonstrated repairs, fictional accounts, profiles, posts and comments. It uses a short-lived JWT in an HttpOnly cookie, server-side permission checks and CSRF protection for authenticated writes.

Source code and setup

<https://github.com/Tejas021/common-mern-security>

Inside this guide

- Pages 2-3: the eight main security lessons.
- Page 4: clone and run the app.
- Page 5: project checklist and optional topics.
- Pages 6-7: free and paid courses, with provider names and links.

Prepared from the seminar materials. Course details checked on 3 October 2026; provider terms and prices can change.

SECTION 02

Identity, permissions and input

1 / Ownership: check who may change a resource

A member editing a profile must be allowed to edit that particular profile. A valid MongoDB ID or a hidden Edit button does not establish permission. Express must check the relationship between the signed-in account and the target record.

Check: the owner can update the profile; another member is denied and the record stays unchanged.

2 / Editable fields: accept only what the feature needs

The profile editor may accept displayName, but must protect role, account ownership and other privileged fields. Build an explicit allowlist or strict schema. Copying an arbitrary request body into a database update gives the client too much control.

Check: permitted fields work; extra fields cannot change privileged account state.

3 / JWTs: verify before trusting claims

Decoding reads a token; verification establishes whether its claims can be trusted. Use a maintained library and explicit checks for the signature, permitted algorithm, issuer, audience and expiry. Then enforce authorization for the requested resource.

Common loads the current account after verification and revokes the current token on logout. Signing keys belong on the server. Do not log tokens, passwords or authentication cookies.

4 / NoSQL input: keep data separate from query structure

A search value intended to be a string must be a bounded string, not an arbitrary JSON object. Construct MongoDB filters and updates from validated values and server-defined structure. Client JSON must not become database instructions.

Check: valid input works; unsupported types fail before database work or protected state changes.

Apply this to your project

For each protected route, identify the caller, the target resource, the allowed action and the accepted fields. Enforce those decisions in the backend and keep a normal success case alongside a meaningful denial case.

SECTION 03

Browser security, abuse and privacy

5 / XSS: render comments as content

Render ordinary comments through React text interpolation. Avoid raw HTML rendering for user content. If rich text is required, use a maintained sanitizer with a deliberate policy for tags, attributes and links. A content security policy adds another layer; it does not replace safe rendering.

6 / CSRF: protect cookie-authenticated changes

Browsers can attach authentication cookies automatically. State-changing routes need appropriate Origin and CSRF checks for their login design. SameSite helps, but its behaviour depends on the deployment. In the seminar, different localhost ports were different origins on the same site.

Common combines Origin checks and CSRF protection. CORS controls which browser origins can read responses; it does not establish permission or replace CSRF protection.

7 / Login abuse: limit repeated attempts

Apply suitable rate limits to login and other sensitive endpoints. Use safe errors and useful monitoring without recording passwords or tokens. Choose limits that preserve legitimate access. A deployed app with several server instances needs coordinated limiter state.

8 / Public responses: return only public fields

A public profile should return a deliberate contract, such as ID, display name and avatar. Private email addresses, password hashes and reset tokens do not belong in that response. Hiding a field in React does not hide it from the browser network response.

Check: comments display safely, legitimate writes work, failed CSRF checks cannot change state, and public responses contain only permitted fields.

The shared lesson

Each control solves a specific problem. HttpOnly does not solve all XSS or CSRF risks. CORS is not authorization. A dependency scan cannot establish that your application permissions are correct.

SECTION 04

Clone and run Common

Clone the Common reference app

<https://github.com/Tejas021/common-mern-security>

Requirements: supported Node.js 22.12 or newer and npm. Initial package installation and the MongoDB download require internet.

First terminal

```
git clone https://github.com/Tejas021/common-mern-security.git
cd common-mern-security
npm ci
npm run build
npm run mongo
```

Second terminal

Keep MongoDB running. Open the same repository folder in another terminal:

```
npm run seed -- --confirm-reset
npm start
```

Open <http://127.0.0.1:4200>. Use 127.0.0.1 to match the app origin policy.

Fictional logins

- hermione@common.demo - member
- ron@common.demo - member
- admin@common.demo - administrator, displayed as Luna
- Password for all three: CommonDemo!2026

Where to look in the code

Start with SECURITY-MAP.txt. auth.mjs covers authentication and CSRF; policies.mjs and routes.mjs enforce permissions; validation.mjs defines input contracts; CommentBody.jsx renders comments; app.mjs configures origins and limits.

Reset and stop

Seeding replaces fictional local data and invalidates old logins. Ctrl+C stops each terminal. Run npm test for the smoke check. The README includes troubleshooting and an optional Docker database setup.

This is local teaching software. Read the README for deployment, transaction and retry limitations before adapting it.

SECTION 05

Project checklist and practice

Use this checklist on one profile, post or order feature in your college project.

- Identity: credentials are verified; expiry and logout follow the design.
- Permissions: protected reads and writes check the caller and target on the server.
- Input: accepted fields, types and lengths are explicit; privileged fields stay protected.
- Database: filters and updates use validated values and trusted structure.
- Browser: comments render safely; cookie writes have appropriate CSRF protections.
- Privacy: responses contain only documented public or permitted fields.
- Abuse: sensitive endpoints have limits and useful, safe monitoring.
- Verification: legitimate behaviour works; denied operations leave protected state unchanged.

Before deployment

Review HTTPS, production cookie settings, server-side secrets, database authentication and network access, supported dependencies, safe errors and proxy/origin assumptions. Coordinate rate-limit state if the app uses multiple instances.

Optional topics from the seminar materials

- Trusted pricing: calculate prices and discounts from server-side data.
- Race conditions: preserve stock and balance rules with atomic operations or transactions; account for retries.
- Link previews: restrict outbound destinations and protocols, consider redirects, and enforce timeouts and size limits.

A simple practice task

Review a profile update route. Explain the identity, ownership rule, accepted fields and public response. Record one successful owner update and one denied non-owner update with no stored change.

Useful official references

OWASP security guidance

<https://cheatsheetseries.owasp.org/>

Express production security

<https://expressjs.com/en/advanced/best-practice-security.html>

SECTION 06

Recommended free courses

Start with a short introduction, then apply a JavaScript or API-security lesson to your own app. Paid courses are optional.

Hacksplaining

Provider: Hacksplaining | Free for personal use

Best quick introduction. Interactive lessons and quizzes explain common web-security mistakes, with examples including JavaScript. Team reporting is a paid feature.

Lessons

<https://www.hacksplaining.com/>

Snyk Learn - JavaScript and API security

Provider: Snyk Learn | Free

Best immediate fit for students already building MERN apps. Choose JavaScript lessons and the OWASP/API security learning paths; examples vary by language.

JavaScript security catalogue

<https://learn.snyk.io/catalog/security-education/javascript/>

Developing Secure Software (LFD121)

Provider: Linux Foundation / OpenSSF | Free

A more structured course on secure design, input/output handling, dependencies and verification. Coding knowledge is expected. Completion certificate requirements are listed by the provider.

Course and enrollment

<https://training.linuxfoundation.org/training/developing-secure-software-lfd121/>

Web Security Academy

Provider: PortSwigger | Free training

Good supplementary web-security explanations and interactive practice. Training is free; professional certification is separate.

Getting started

<https://portswigger.net/web-security/getting-started>

More foundation options

Cisco Networking Academy: Introduction to Cybersecurity - free beginner foundation. Harvard CS50: Introduction to Cybersecurity - free learning, with optional paid edX verification. Full Stack Open, University of Helsinki - free React/Node development course; useful for strengthening the stack, rather than dedicated security training.

Cisco course

<https://www.netacad.com/courses/introduction-to-cybersecurity>

Harvard CS50 course and certificate FAQ

<https://cs50.harvard.edu/cybersecurity/faqs/>

Full Stack Open

https://fullstackopen.com/en/part0/general_info/

SECTION 07

Optional paid courses

Buy only if the course format suits you. Check preview lessons, current examples and checkout price; Udemy prices vary by region and sale.

OWASP Top 10 2025: Web App Security for Beginners (No Code)

Platform: Udemy | **Instructor:** Soerin Bipat | **Paid**

A short conceptual introduction, about 1 hour 38 minutes. Suitable for beginners who want the vocabulary before implementation; some material follows earlier OWASP versions.

Udemy course

<https://www.udemy.com/course/web-application-security-for-absolute-beginners-no-coding/>

Application Security - The Complete Guide

Platform: Udemy | **Instructor:** Derek Fisher | **Paid**

About 12 hours 33 minutes on OWASP, threat modelling and security through development. Choose it for broader application-security design; it is not specifically a MERN course.

Udemy course

<https://www.udemy.com/course/application-security-the-complete-guide/>

Node.js, Express, MongoDB & More: The Complete Bootcamp

Platform: Udemy | **Instructor:** Jonas Schmedtmann | **Paid**

About 42 hours of backend project work, with authentication and security sections. A development companion rather than a dedicated security course. It includes Pug and older runtime examples; check current documentation when adapting them.

Udemy course

<https://www.udemy.com/course/nodejs-express-mongodb-bootcamp/>

Our recommendation

Begin with Hacksplaining and Snyk Learn. Choose LFD121 for deeper structured study. For a paid option, pick Soerin Bipat for a brief introduction or Derek Fisher for broader design. Jonas is useful when you also need stronger Node/Express foundations.

Sources and scope

The seminar summary follows the Common seminar materials. Extra pricing, concurrency and link-preview topics are optional extensions. Course recommendations use provider curricula and access terms; they are not claims that we completed each course. Exact Indian checkout prices were not verified.